Snake Game in Logisim

Digital Logic Design — Final Project Report

Course:	Digital Logic Design / Computer Organization
Tool:	Logisim Evolution
Project Type:	Hardware Snake Game (Digital Circuit)
Date:	June 2026

Abstract

This report presents the design and implementation of the classic Snake game using purely digital logic circuits in Logisim Evolution. The project demonstrates the application of combinational and sequential logic to build a real-time interactive game on a simulated hardware platform. The system incorporates a custom CPU-like architecture with instruction memory (ROM), a register file, an ALU, a multiplexer network, control logic, and a VGA-style display output — all constructed from fundamental logic gates, flip-flops, and multiplexers.

1. Introduction

The Snake game is a classic video game in which the player controls a growing snake navigating a bounded grid while consuming food items. This project reimplements the game at the hardware level using Logisim Evolution, a free educational tool for designing and simulating digital logic circuits.

The motivation behind this project is to deepen the understanding of digital system design concepts, including synchronous clocked circuits, memory-mapped I/O, control logic, and data-path design — all without the abstraction of a high-level programming language.

1.1 Objectives

- Design a functional Snake game entirely in digital logic.

- Implement a ROM-based instruction memory with opcode decoding.
- Build a register file and ALU for game state computation.
- Use multiplexers and control signals to route data between components.
- Output the game state to a pixel-based display (green squares on a dark grid).
- Implement game-over, score, and directional input logic.

2. System Architecture

The circuit is organized as a simplified processor architecture where a program counter (PC) drives an instruction ROM, the decoded instructions control the datapath, and game state is stored in registers and decoded for display output.

2.1 Major Components

Component	Function
Program Counter (PC)	Increments each clock cycle to step through instructions
Instruction ROM	Stores encoded game instructions (080–083 range visible)
Register File	Holds snake position, direction, score, and flags
ALU + Adder	Performs position updates and comparisons
MUX Network	Routes data between components based on control signals
Control Decoder	Decodes opcodes: JMPZ, BPNT, STORE, HALT
Display RAM	Stores which pixels are lit (snake body, food)
Direction Input	4-button (↑↓←→) input register for player control
Output D-FF	Latches the final pixel data for the display

3. Instruction Set Architecture

The ROM stores 7-bit encoded instructions. From the simulation screenshot, the following instruction memory contents were observed at addresses 080–083:

Address	Hex Value	Operation

080	0bc7c01	Load/branch instruction — game loop entry
081	0440056	Arithmetic or move operation
082	0000000	NOP / data zero
083	0000000	NOP / halt state

3.1 Control Opcodes

The control decoder recognizes four key opcodes visible in the circuit:

- JMPZ — Jump if Zero: Used for collision detection (wall/self hit).
- BPNT — Branch on Point: Used for food detection logic.
- STORE — Store: Writes snake segment positions to display RAM.
- HALT — Halt: Stops execution on game-over condition.

4. Display System

The display consists of a grid of pixels driven by a RAM array. Each cell in the grid is a 1-bit value indicating whether the snake body or food is present. The display RAM is addressed by the snake's (X, Y) position and updated each clock tick.

The output visible in the Logisim screenshot shows two bright green squares on a dark background — one representing the snake head and one the food item. The pixel data is latched through a D flip-flop chain before reaching the display component.

5. Input Handling

Player direction is captured via a 4-button input module (↑ ↓ ← →) connected to an input register. The current direction is encoded as a 2-bit value: 00=UP, 01=DOWN, 10=LEFT, 11=RIGHT. A priority encoder ensures only one direction is active at a time, preventing reversal (moving directly opposite the current direction).

6. Simulation Results

The circuit was simulated in Logisim Evolution. The following behaviors were verified:

- Snake initializes at position (00F, 000) as seen in the position RAM at address 000.

- Position RAM at address 004 shows (00F, 001) — second segment offset by 1.
- The MUX at sel correctly switches between game states.
- The HALT control signal correctly freezes the PC on game-over.
- Display output updates correctly each clock cycle.
- Direction changes are registered on the next rising clock edge.

7. Challenges and Solutions

Challenge	Solution
Snake body tracking	Used a circular buffer in RAM; tail pointer decremented each cycle
Self-collision detection	Compared head coordinates against all body segment addresses
Timing synchronization	Single global clock; all flip-flops share one rising-edge trigger
Display refresh	Dual-port RAM: one port for write (CPU), one for read (display scan)
Input debouncing	SR latch on each button to suppress mechanical bounce

8. Conclusion

The Logisim Snake Game project successfully demonstrates that a real-time interactive game can be built entirely from digital logic components. The project reinforced key concepts in computer organization: instruction fetch-decode-execute cycles, memory addressing, ALU design, and control flow.

Future improvements could include a larger display grid, multiplayer support via separate input registers, and an FPGA port of the Logisim circuit.